

RAG-Based AI Chatbot for PDF Document Querying and Management

Project Objective

The goal of this project is to develop an AI-powered chatbot system that allows users to upload, manage, and query large PDF documents (approx. 3GB total). Users will receive precise, context-aware responses drawn directly from the selected PDF using Retrieval-Augmented Generation (RAG).

Core Features

- 1. PDF Upload and Processing**
 - Upload single or multiple PDFs via web UI
 - Extract and clean text using pdfplumber
 - Chunk and index text for semantic search
- 2. Automatic Dropdown Creation**
 - Each uploaded file automatically appears in a dropdown menu
 - Users select a specific file to scope their questions
- 3. Question Answering**
 - User queries are converted into embeddings
 - Top relevant chunks retrieved using **FAISS**
 - Context-aware answers generated via **Gemma 3:27B LLM**
- 4. PDF Management**
 - **Delete** option to remove PDF, embeddings, and dropdown entry
 - **Edit** option to replace existing PDF and re-index content
- 5. Frontend Interface**
 - Built with **HTML, CSS, Bootstrap, JavaScript**
 - Intuitive UI for upload, dropdown selection, and chatting
- 6. Backend System**
 - Developed using **FastAPI** (async)
 - Local file storage and FAISS vector DB
 - Optional metadata tracking with SQLite/PostgreSQL

Technologies Used

Component	Technology
Frontend	HTML, CSS, JavaScript, Bootstrap
Backend API	FastAPI (Python, async)
PDF Processing	pdfplumber
Vector DB	FAISS (Local)
LLM	Gemma 3:27B
Orchestration	LangChain or custom
File Storage	Local Disk
Embeddings	Instructor-XL / Sentence Transformers
Metadata DB	SQLite / PostgreSQL (optional)

System Workflow

- Upload PDF** → Text Extraction → Text Chunking
- Generate Embeddings** → Store in FAISS
- Dropdown Creation** (auto)
- Query Input** → Embedding → Search → Generate Answer
- Response Displayed via Chat UI**
- Manage PDFs** (Edit/Delete with full sync)

Optimization Strategy

- Asynchronous FastAPI endpoints for speed
- Streaming responses from LLM
- Embedding caching and pre-indexing
- Text compression before embedding
- FAISS quantization for large-scale vector search

Milestones & Timeline

Phase	Tasks Covered	Est. Time
Research & Setup	Environment, model check, PDF processing, vector DB testing	1-2 days
Backend Development	Upload, extract, chunk, embed, FAISS search, API endpoints	7-8 days
Frontend Development	Bootstrap UI, dropdown logic, file manager, chat interface	1 day
LLM Integration (Gemma 3:27B)	Context passing, streaming replies, response formatting	1-2 days
Testing & Optimization	Scale tests with 3GB PDFs, speed optimization, bug fixes	2-3 days
Deployment	Host on server (Runpod),	2-3 days

Deliverables

- Fully functional PDF RAG chatbot system
- Admin-friendly upload/edit/delete interface
- Real-time semantic query capabilities
- Deployment guide